LLM ENGINEERING

Week 04

Modern Attention

Bethgelab · Thursday, 5 November 2026

University of Tübingen · Winter term 2026/27

Tübingen AI Center



Announcements

Next week, 12 November

- Week 5: Optimization

Quiz 2 on ILIAS

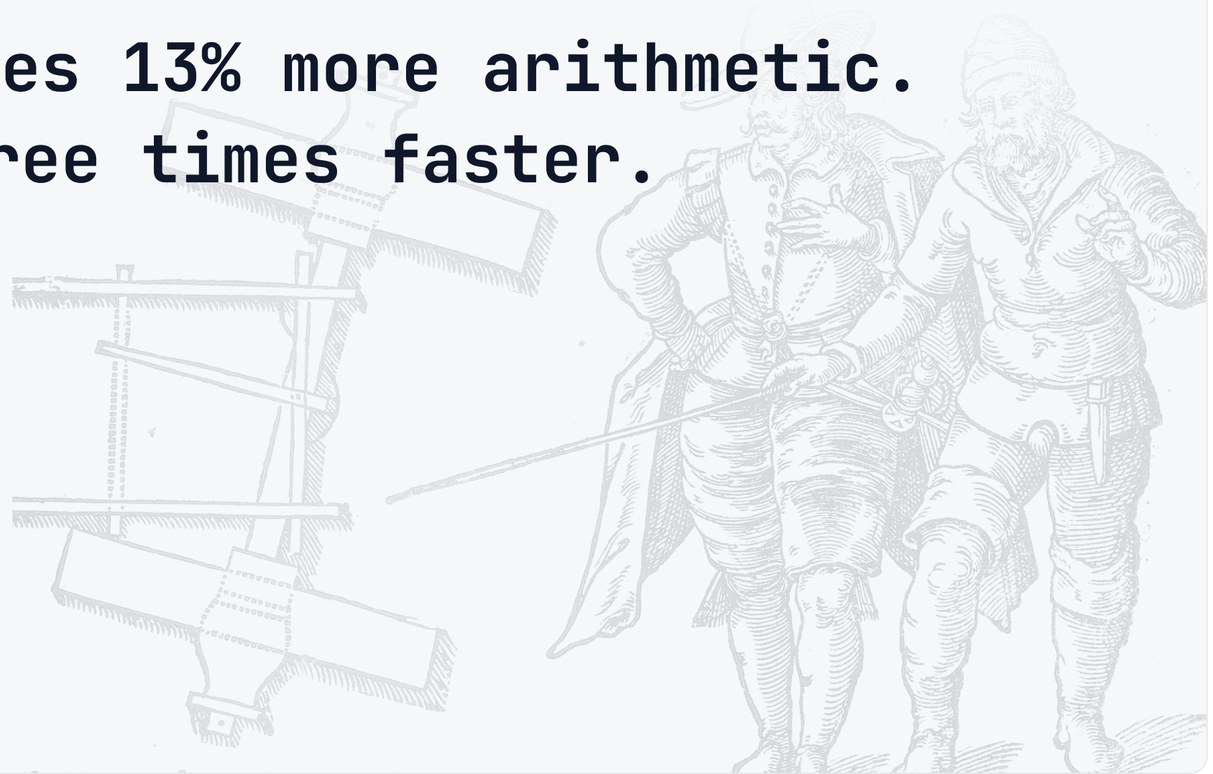
- Weeks 3, 4 and Bonus 2
- Due Friday 13 November, end of day

Third exercise: this lecture

- Due Thursday 12 November, 10:00
- Email when it reaches your repository

How can more FLOPs run faster?

FlashAttention does 13% more arithmetic.
It runs three times faster.



What limits attention on a GPU, and how do models cut it?

- Roofline: is compute or memory traffic the limit?
- **FlashAttention**: exact attention, yet faster
- Sliding-window attention (SWA): recent tokens only
- **Multi-head latent attention** (MLA): less per token
- Exercise: build SWA and MLA, measure them

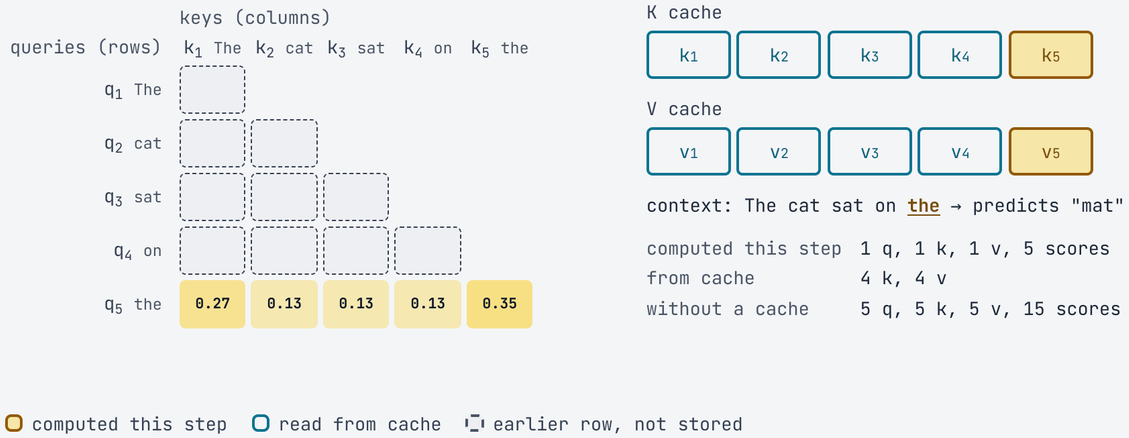
Where Week 3 left off

- GQA: query heads share KV heads
- Masks decide which keys count
- RoPE rotates queries and keys, NoPE adds no position
- KV cache: keys and values of every past token and layer

Position and locality: separate choices

Decode reads every cached key and value

ILLUSTRATIVE | Prefill | Decode 1 | Decode 2



context: The cat sat on **the** → predicts "mat"

computed this step	1 q, 1 k, 1 v, 5 scores
from cache	4 k, 4 v
without a cache	5 q, 5 k, 5 v, 15 scores

computed this step | read from cache | earlier row, not stored

- Decode: one new query row
- Reads every cached key and value
- Cache grows by one K, V per token

Fast memory is small, large memory is slow

GPU SM

H100 SXM: 132 SMs · 50 MB L2 · 80 GB HBM3



Registers
256 KB per SM
bandwidth not given · a few ns

Shared memory
up to 228 KB per SM
31 TB/s peak, whole GPU · about 10–20 ns

L2 cache
50 MB
12 TB/s peak · about 150–200 ns

HBM3
80 GB
3.35 TB/s · about 400 ns

Hover over a part, or tab to it. Click an SM to open it.

After NVIDIA, 2026b, Figures 3 and 4

- 132 streaming multiprocessors (SMs) share one L2 cache
- On-chip static RAM (SRAM): registers, shared memory
- Off-chip high-bandwidth memory (HBM): 80 GB, 3.35 TB/s

Every kernel computes and moves bytes

resource lower bound

$$T \geq \max\left(\frac{F}{P_{\text{peak}}}, \frac{B}{\beta}\right)$$

- Compute time: FLOPs ÷ peak
- Movement time: bytes ÷ bandwidth
- The slower one binds

Arithmetic intensity counts FLOPs per byte

arithmetic intensity

$$I = \frac{F_{\text{useful}}}{B_{\text{HBM}}}$$

- Bytes to and from high-bandwidth memory (HBM)
- Low **intensity**: bandwidth binds
- High intensity: compute can bind

An H100 SXM, in BF16:

How many FLOPs per byte keep it busy?
About 3, 30, 300 or 3,000?

An H100 SXM needs about 300 FLOPs per byte

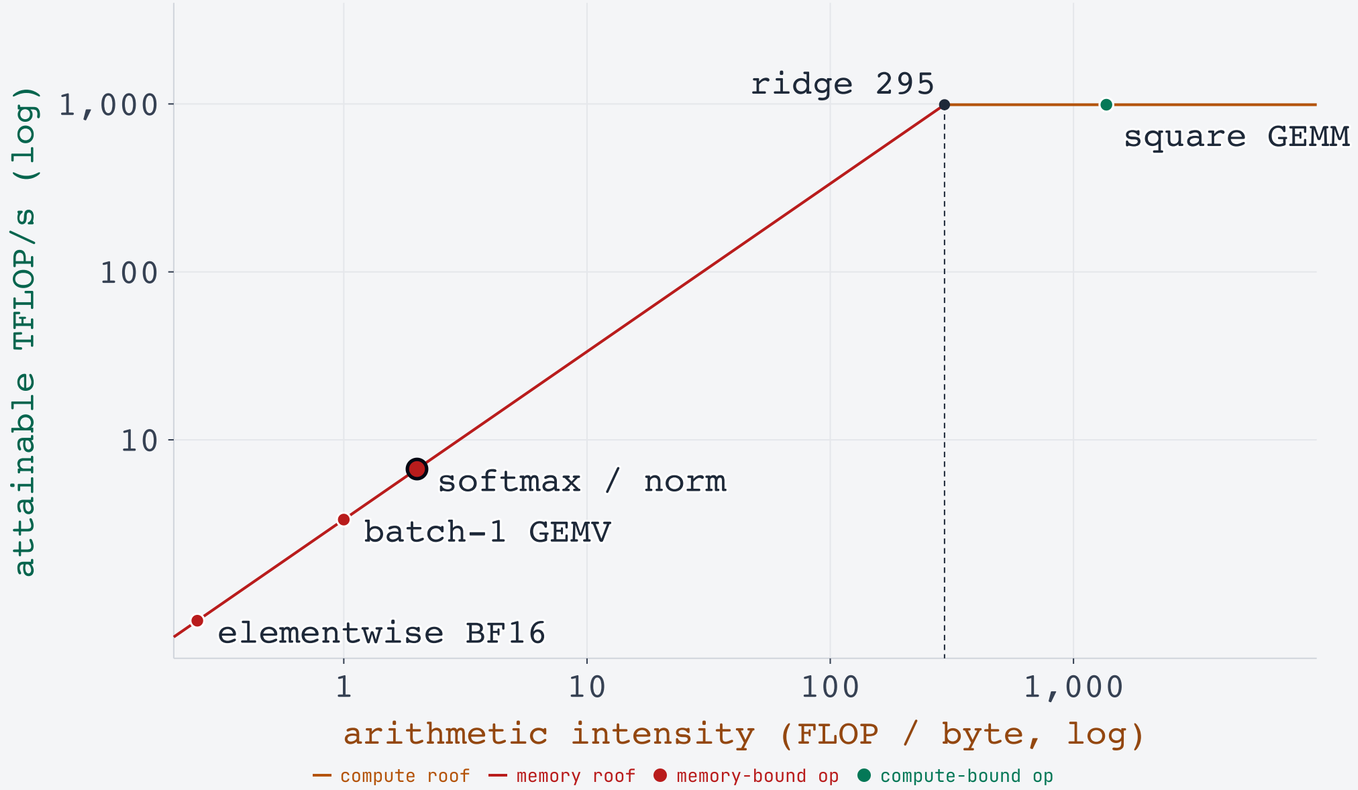
attainable throughput

$$P_{\text{attainable}} = \min(P_{\text{peak}}, I \times \beta_{\text{HBM}})$$

H100 SXM theoretical reference; profile the assigned GPU for the exercise

workload: softmax / norm 2 FLOP/byte · memory-bound

peak ≈ 989.5 TFLOP/s (dense bf16) | bandwidth ≈ 3.35 TB/s | ridge ≈ 295 FLOP/byte



The H100 SXM is our reference GPU

H100 SXM reference ridge

$$I_{\text{ridge}} = \frac{989.5 \times 10^{12}}{3.35 \times 10^{12}} = 295.37\ \text{FLOP/byte}$$

- HBM3: 80 GB at 3.35 TB/s (NVIDIA, 2026a) (NVIDIA, 2026b) (NVIDIA, 2026c)
- BF16: 989.5 TFLOP/s dense, 1,979 only with 2:4 sparsity
- Profile your assigned GPU

Operation intensity depends on the traffic model

GEMM, each tensor crosses HBM once

$$I_{\mathrm{GEMM}} = \frac{2MKN}{b\,(MK + KN + MN)}$$

- Elementwise, BF16: 0.25 FLOP/byte
- Matrix-vector at batch 1: about 1, weights dominate
- Softmax, norm: about 2, depends on passes and fusion
- Square matmul, n = 4,096: 1,365
- Only the matmul clears the ridge of 295

Model FLOPs utilization (MFU) compares model work with peak

single-device MFU

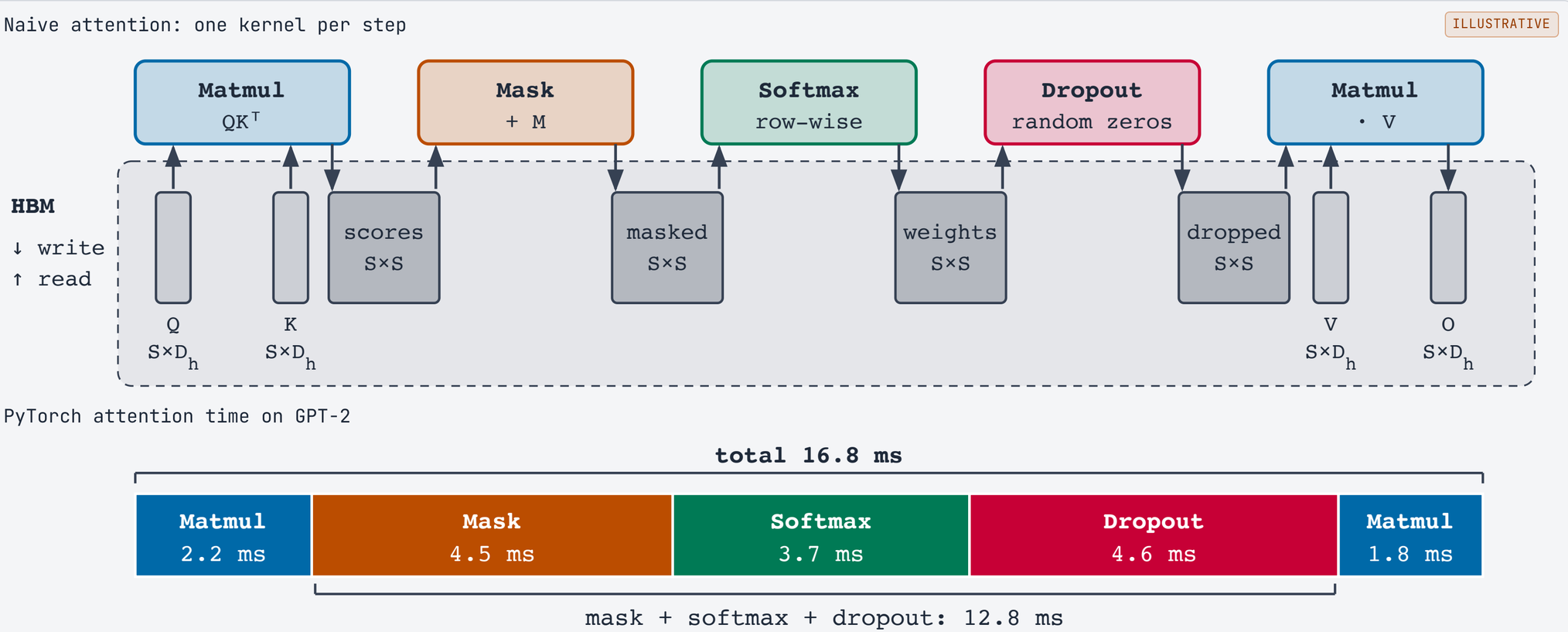
$$P_{\text{model}} = F_{\text{model/token}} \times R_{\text{token}}; \quad \text{MFU}_{\text{1GPU}} = \frac{P_{\text{model}}}{P_{\text{peak,matching}}}$$

- Model FLOPs utilization (MFU), from PaLM (Chowdhery et al., 2023)
- 80 GFLOP/token × 8,000 tokens/s
- = 640 TFLOP/s, 64.68% of the 989.5 peak
- Covers the whole model end to end

Naive attention is memory-bound here

- S × S scores written to HBM once, read twice
- S = 4,096, 32 heads: 3 GiB moved
- 85 FLOP/byte: below the ridge

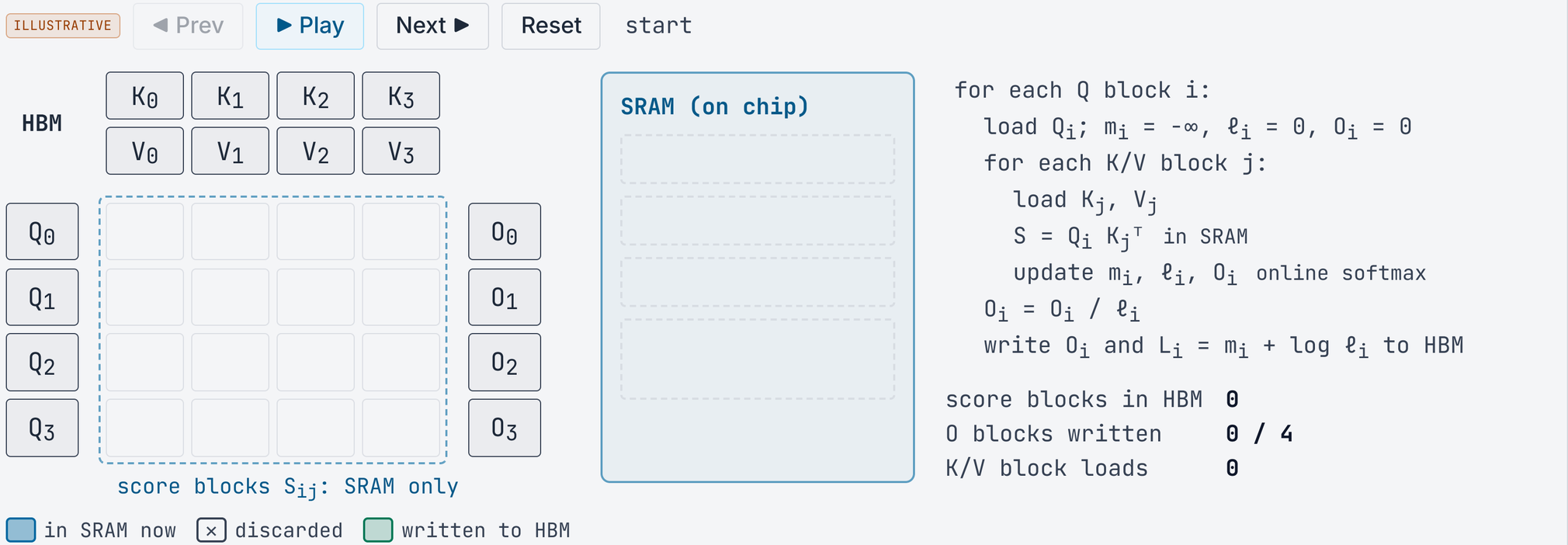
Naive attention mostly moves S × S tensors



Dao et al., 2022

- One kernel per step, each writing to HBM and reading back
- Mask, softmax, dropout: 12.8 of 16.8 ms (Dao et al., 2022)

Three ideas make FlashAttention work



- Tiling: K/V blocks stream through on-chip static RAM (SRAM) (Dao et al., 2022)
- Online softmax: running max `m`, sum `ℓ`, output `o`
- Backward: recompute blocks from a row statistic

Exact attention without the score matrix

FA1 I/O result

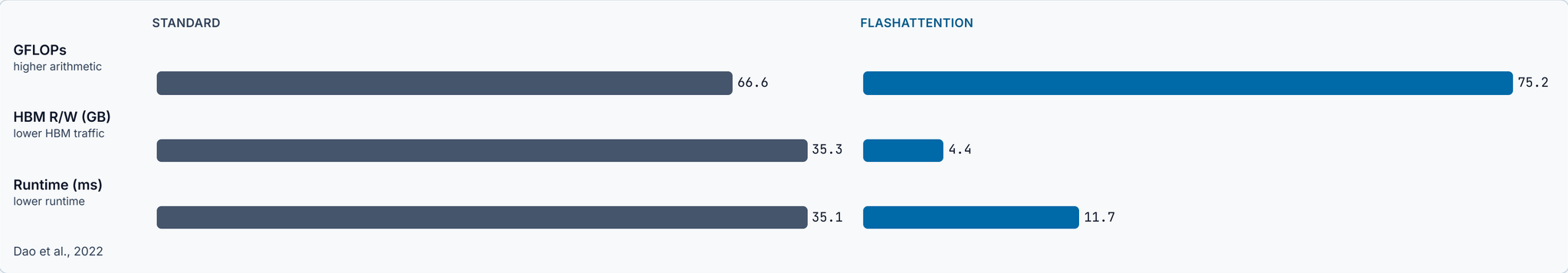
$$\begin{aligned} \text{FlashAttention-1:} \quad & \Theta(N^2 d^2 / M) \\ \text{standard attention:} \quad & \Theta(Nd + N^2) \end{aligned}$$

- `N` sequence length, `d` head width, `M` SRAM size (Dao et al., 2022)
- FlashAttention-2's loop order keeps this count
- Counts HBM accesses, not runtime

Back to the puzzle.

13% more FLOPs, yet three times faster: where did the time go?

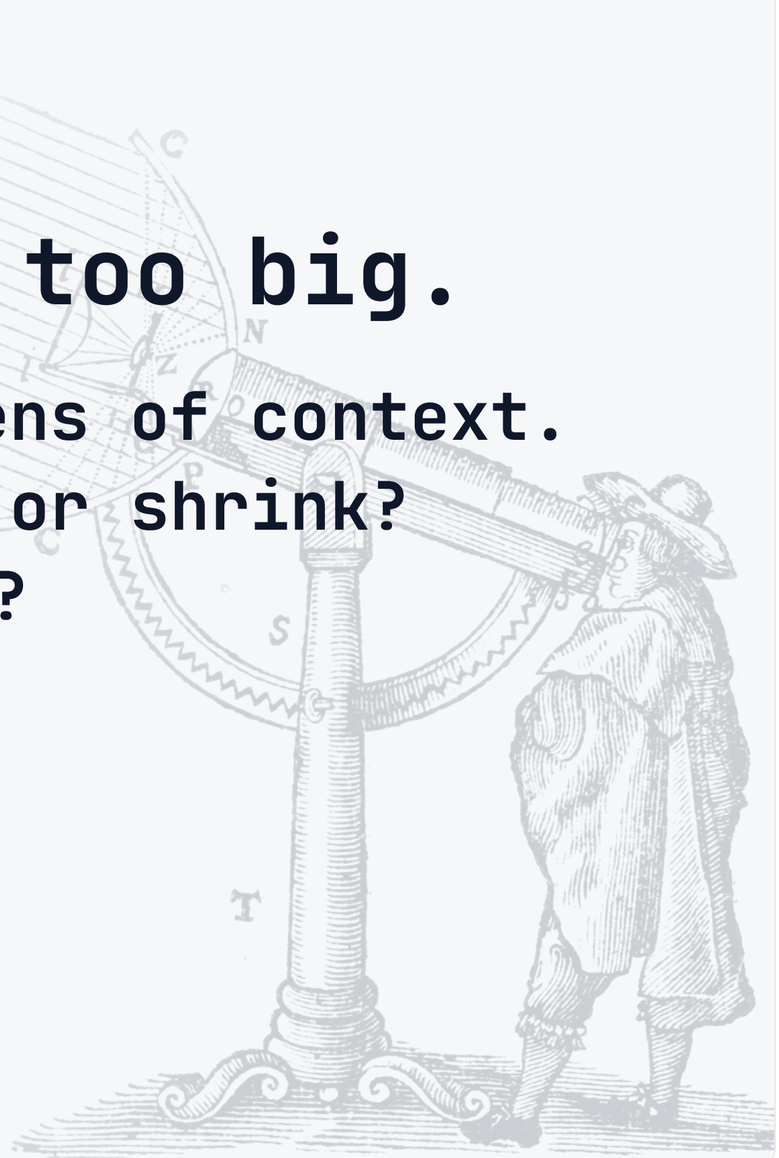
Fewer bytes beat extra arithmetic



- HBM traffic: 35.3 → 4.4 GB (Dao et al., 2022)
- Runtime: 35.1 → 11.7 ms
- Recomputing costs some FLOPs and saves far more traffic

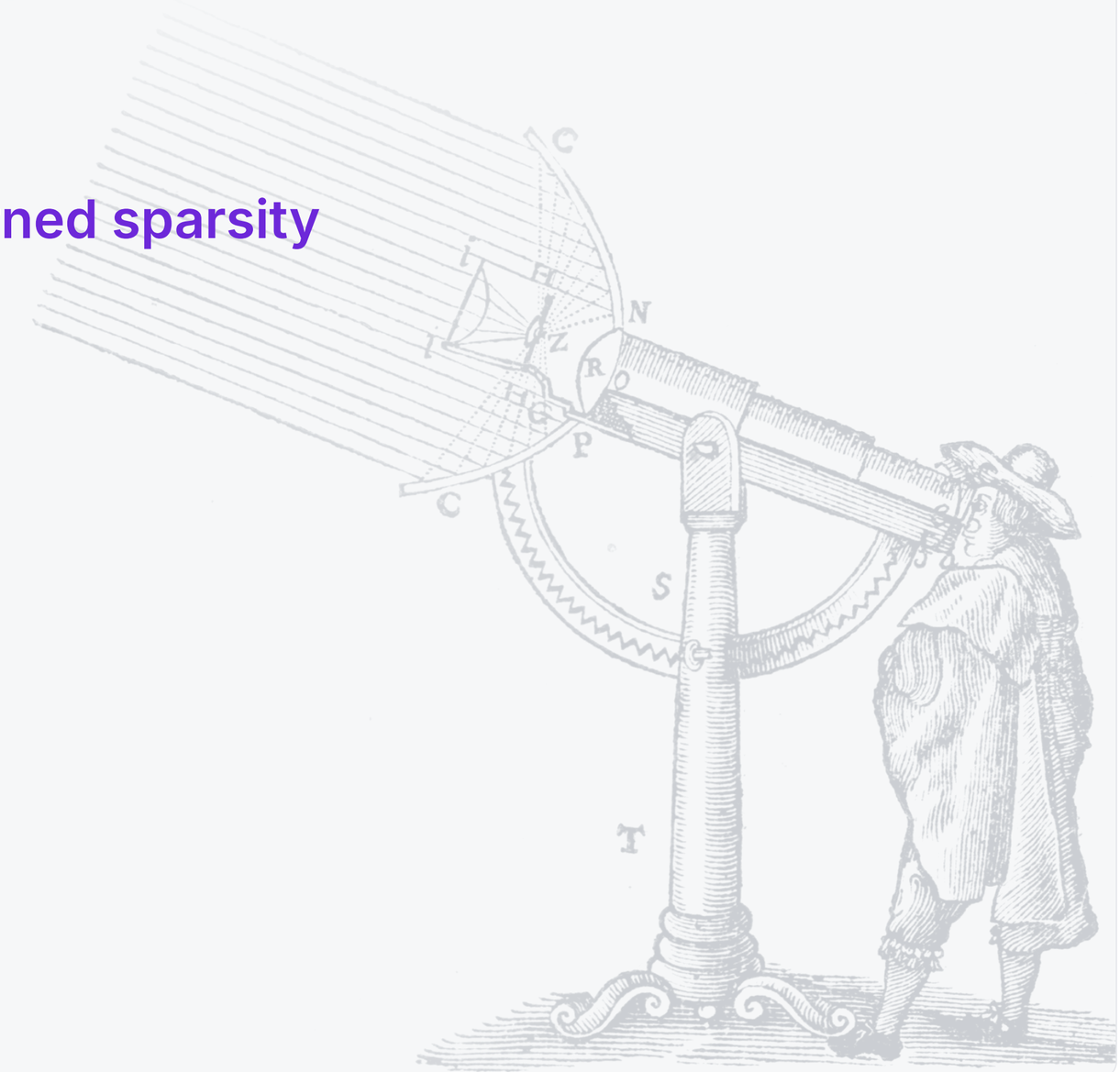
Your KV cache is 8× too big.

You serve a model at 128k tokens of context.
What would you throw away or shrink?
What might break?



Three ways to keep or read fewer bytes

- Drop old tokens: **sliding window**
- Store less per token: MLA
- Read only selected tokens: **learned sparsity**



The Banded Mask, at Last

inclusive SWA boundary

$$\max(0, i - w + 1) \le j \le i$$

- `w = 1`: each token attends only to itself
- `w ≥ S`: full causal attention
- Mistral 7B: `w = 4,096` (Jiang et al., 2023)

Depth Restores Reach

stacked receptive field

$$R_l = 1 + l(w - 1)$$

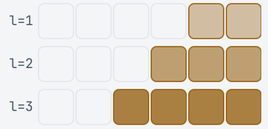
- Reach is capped by the available prefix

ILLUSTRATIVE Window: w=1 w=2 w=3 full

	The	cat	sat	on	the	mat
The	1.00					
cat	0.40	0.60				
sat		0.40	0.60			
on			0.40	0.60		
the				0.40	0.60	
mat					0.40	0.60

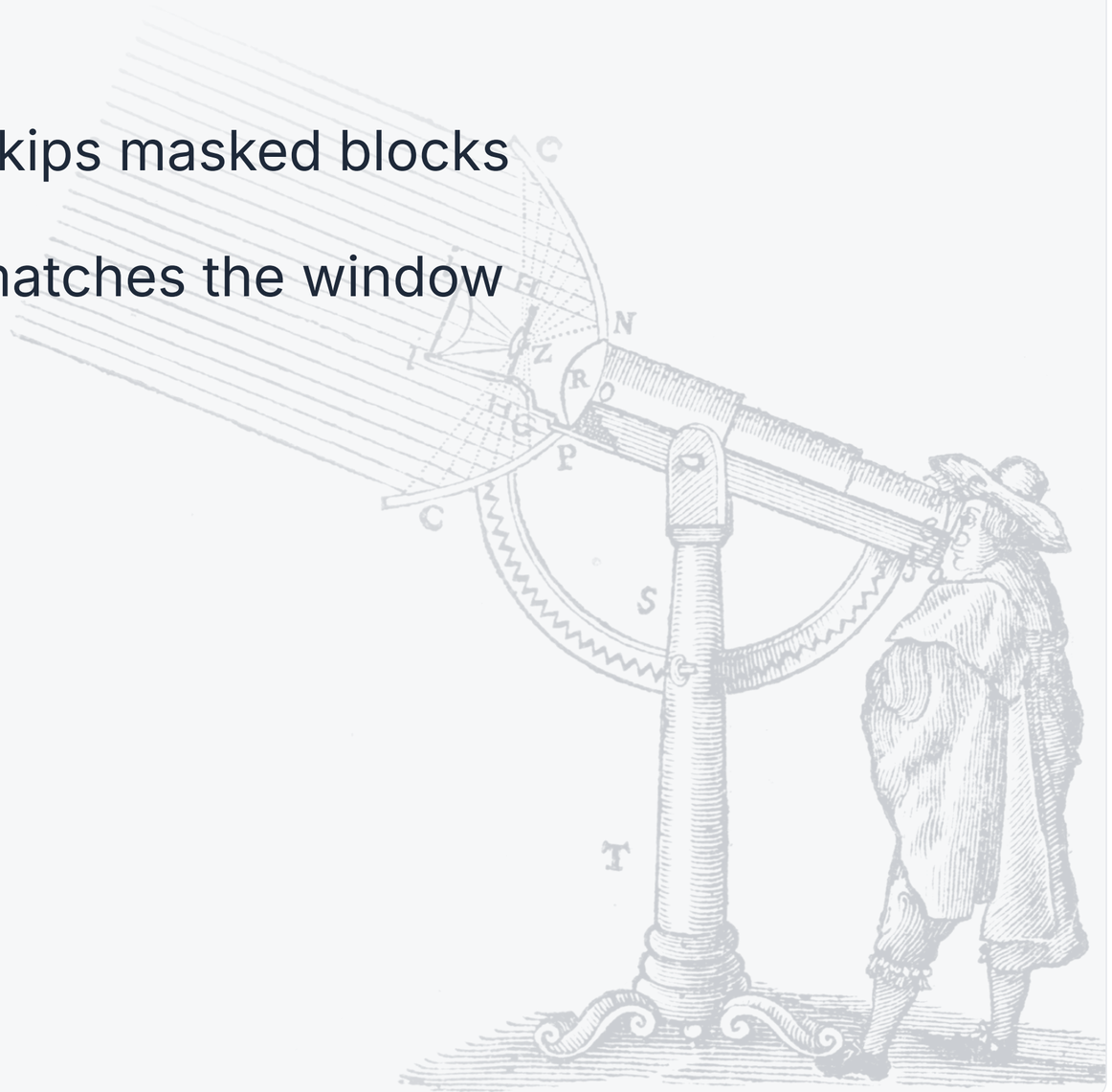
max(0, i - w + 1) ≤ j ≤ i
receptive-field span after l layers: 1 + l(w - 1), capped by the prefix

Receptive field through depth:

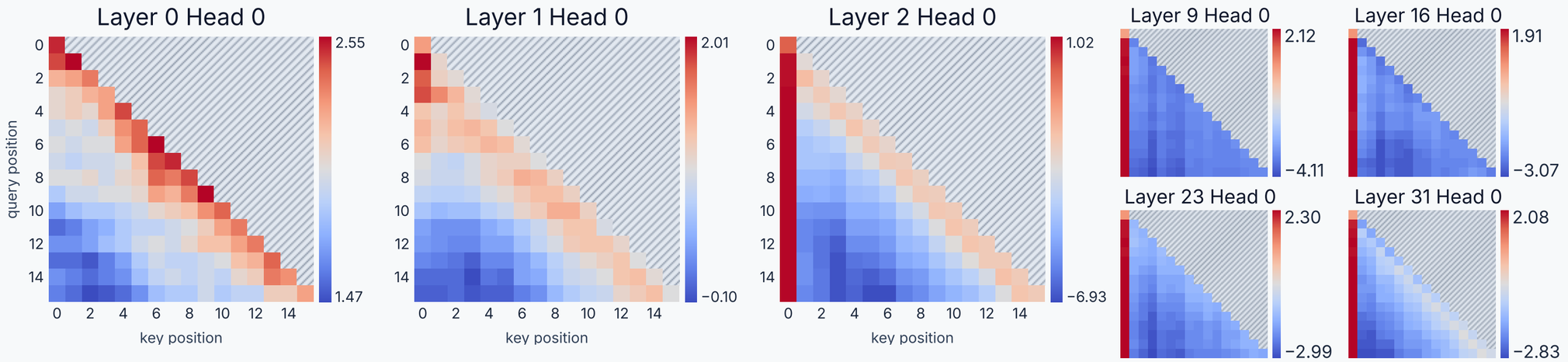


Mask Semantics, Kernel Work, and Cache Residency Are Separate

- Mask: decides which keys count
- `O(Sw)` work needs a kernel that skips masked blocks
- `O(w)` cache needs eviction that matches the window



Some heads attend heavily to the first token



After Xiao et al., 2024, Figure 2 (values approximate)

- Llama-2-7B: early layers local (Xiao et al., 2024)
- Later layers: high logits on the first token

Softmax must put its weight somewhere

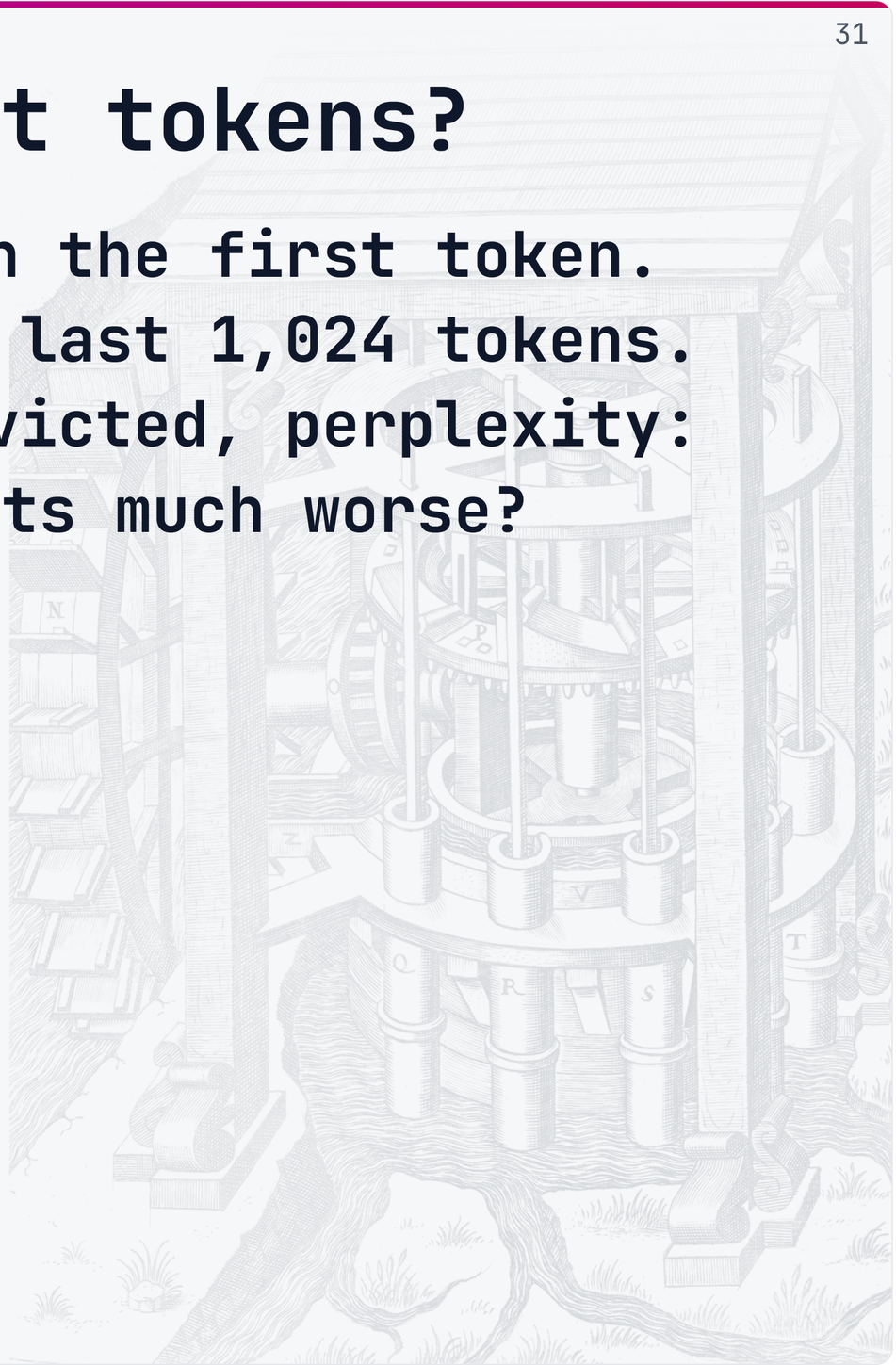
normalization constraint

$$\sum_j \mathrm{softmax}(s)_j = 1$$

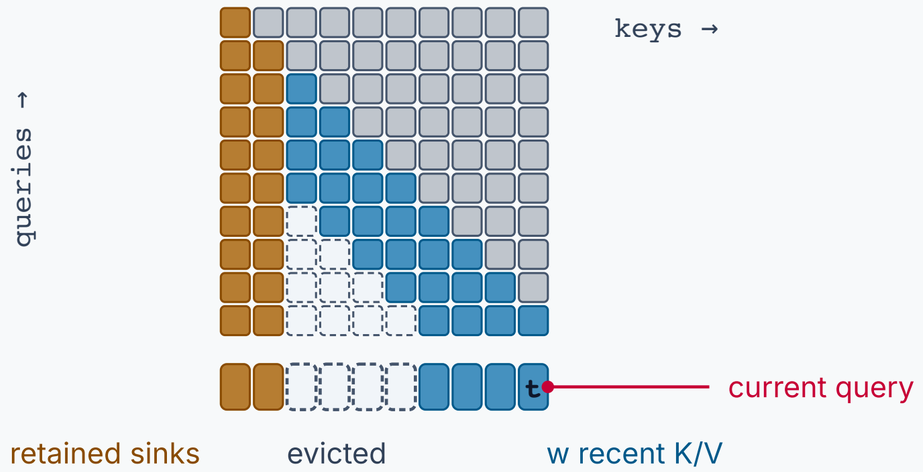
- Weights must sum to one
- An always-visible, low-impact position becomes a **sink**
- Proposed explanation, still a hypothesis (Xiao et al., 2024)

Evict the first tokens?

Some heads park weight on the first token.
A rolling cache keeps the last 1,024 tokens.
Once the first ones are evicted, perplexity:
barely changes, or gets much worse?



Evicting the first tokens breaks a rolling window



ILLUSTRATIVE Adapted from Xiao et al., 2024

- Window alone: perplexity 5,158 (Xiao et al., 2024)
- Keep 4 first tokens: 5.40
- Llama-2-13B, 65K-token PG19 book

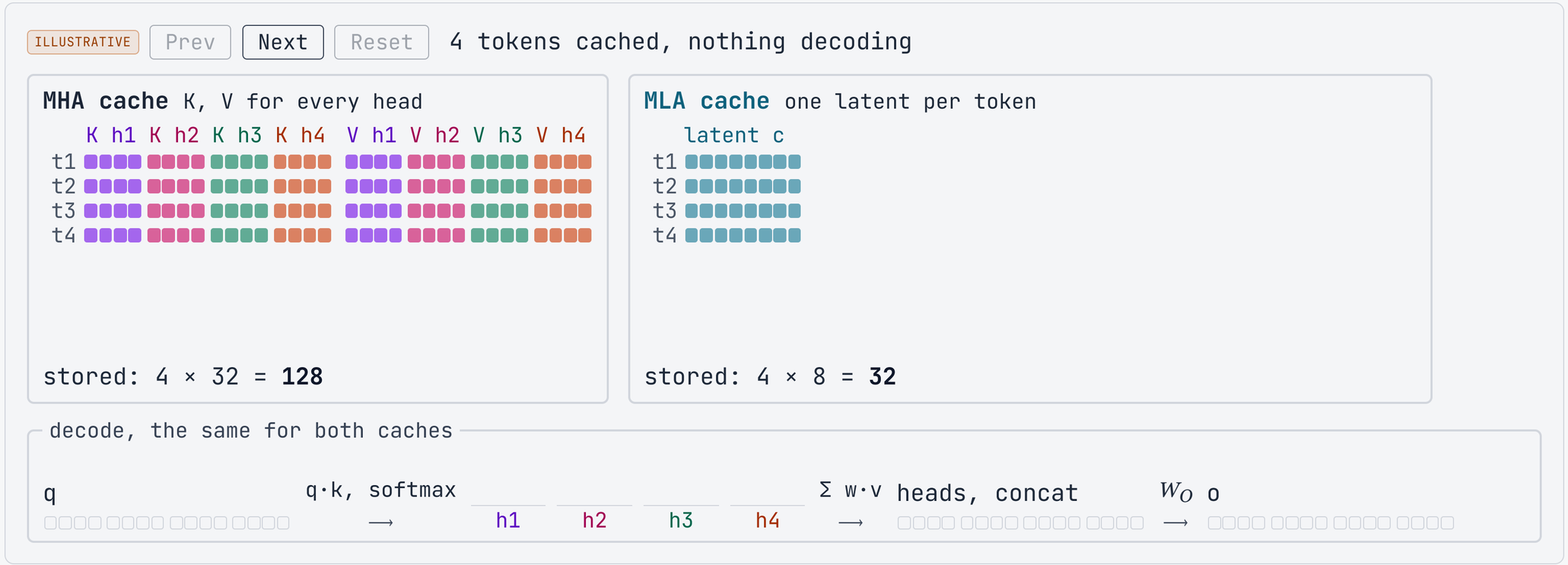
A learned sink bias is not a retained token

learned no-token sink

$$a_j = \frac{e^{s_j}}{e^{b_h} + \sum_k e^{s_k}}, \qquad a_{\text{sink}} = \frac{e^{b_h}}{e^{b_h} + \sum_k e^{s_k}}$$

- Learned per-head scalar `b_h` (OpenAI, 2025)
- Adds no key, value or position to the cache
- Weights on tokens now sum to less than one

Mechanics: One Shared Latent



shared latent and per-head reconstruction

$$c_t^{KV} = W^{DKV} h_t, \qquad k_{t,h}^{C} = W_h^{UK} c_t^{KV}, \qquad v_{t,h}^{C} = W_h^{UV} c_t^{KV}$$

- Cache one latent per token, each head rebuilds K, V (DeepSeek-AI, 2024)

The RoPE Conflict: Rotation and Up-Projection Do Not Commute

non-commutation

$$\mathrm{RoPE}_t(W_h^{UK} c_t^{KV}) \neq W_h^{UK}\,\mathrm{RoPE}_t(c_t^{KV})$$

- RoPE depends on position `t`
- Rotating the latent ≠ rotating each key
- So position needs its own branch (DeepSeek-AI, 2024)

Decoupled RoPE adds one shared rotary key

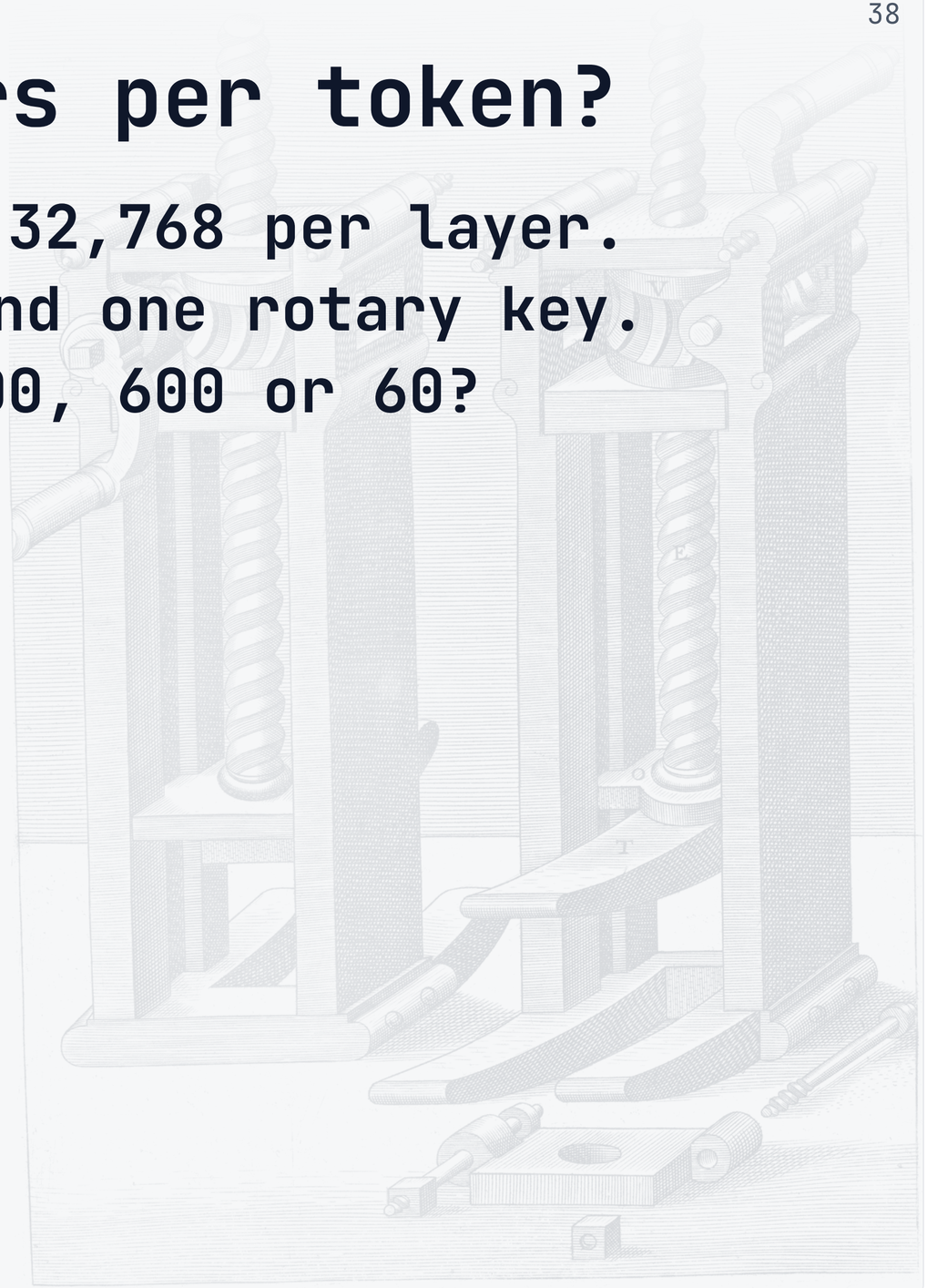
unabsorbed MLA QK paths

$$q_{t,h} = [q^C_{t,h}; \mathrm{RoPE}_t(W^{QR}_h h_t)]$$
$$k^R_t = \mathrm{RoPE}_t(W^{KR} h_t)$$

- Rotary key: `d_r` wide, shared by all heads
- Query: content part plus rotary part
- `d_r` is even because RoPE rotates coordinate pairs

How many numbers per token?

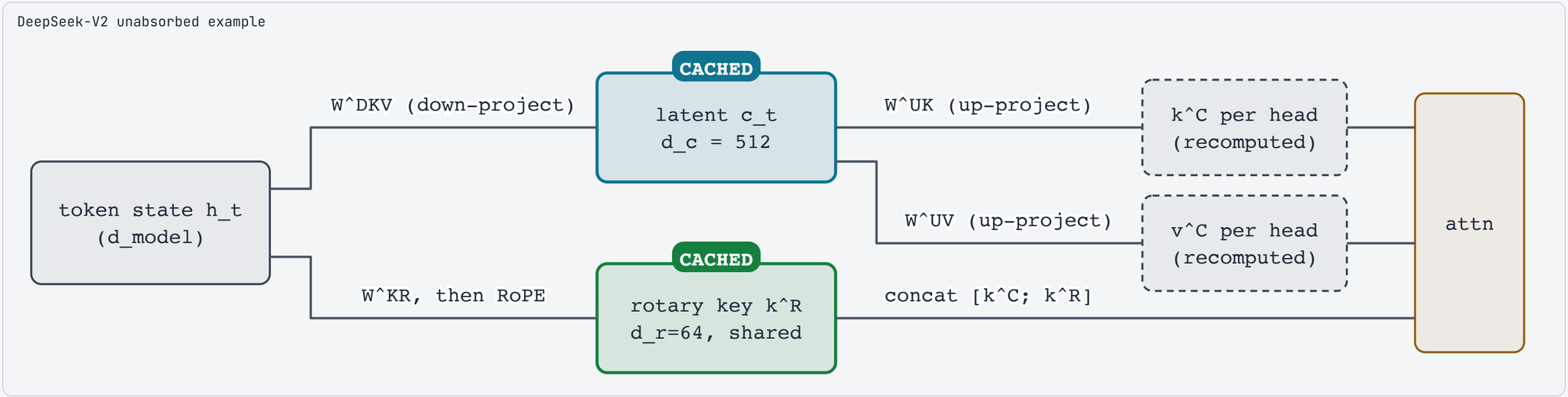
Full MHA here caches 32,768 per layer.
MLA caches a latent and one rotary key.
About 8,000, 2,000, 600 or 60?



MLA caches 576 numbers, not 32,768

stored elements per token

$$2(128)(128) = 32768$$
$$512 + 64 = 576$$
$$1 - 576/32768 = 98.24\%$$



- 98.24% fewer than the MHA comparator (DeepSeek-AI, 2024)
- Counted elements, not measured memory

Learned sparsity chooses what to read

- Window: keys chosen by position
- Learned sparsity: keys chosen per query
- Selecting fewer keys does not shrink the cache

index → select → read → attend (Native Sparse Attention lineage)

1 **INDEX**
a lightweight scorer rates the relevant history

→

2 **SELECT**
keep the k highest-scoring entries

→

3 **READ**
gather the selected K/V state from memory

→

4 **ATTEND**
run exact attention over the selected set

DSA
DSA: shared token top-k, then exact selected attention

MSA
MSA: group-specific contiguous-block top-k, then exact block attention

What limits attention, and how models cut it

1. Roofline finds what binds, MFU compares with a matching peak
2. FlashAttention: same function, fewer bytes, extra FLOPs
3. Window: less reach, savings need skipping and eviction, sinks help
4. Store less (MLA, paid in rebuilds) or read less (DSA, MSA, paid in indexing and recall)
5. Measure the bottleneck: no mechanism wins by name

To-dos until next week

Muddiest point

Do it right now! (2 min)

Before the tutorial

- Write down one question to discuss
- Think about the reflection questions

Self-study

- Read the reading slides and notes

Deadlines

- Quiz 2: Friday 13 November
- Third exercise: Thursday 12 November, 10:00

Reflection questions

1. Recalculate the H100 SXM ridge using decimal units. Which conclusion changes if a sparse peak is paired with dense attention?
2. A square BF16 GEMM has idealized intensity `2n/(3b)`. Which HBM-transfer and reuse assumptions make that expression optimistic?
3. Under the inclusive SWA convention, what is the maximum receptive field of 24 layers with `w=512`? Which two additional implementation choices are required for work and cache savings?
4. Why does the prepared retained-prefix intervention support a cache-policy result without proving that every softmax head needs a sink?
5. For `n_kv=8`, `d_h=128`, `d_c=512`, and `d_r=64`, compare GQA and MLA stored cache elements. Why is attended-token count a different axis?
6. Compare DSA and MSA on selection granularity, sharing, bytes read, index metadata, and kernel shape. Why is 2,048 not a general design constant?

